

bending.forks.swe@gmail.com

Norme di progetto

Capitolato C3: Fossdot – Smart Speaker Open Source

Responsabile	Nicholas Salmaso	**Versione**	0.1.0
Redattori	Marco Beltrame	**Tipo**	Documento Interno
	Lorenzo Milanese	**Destinatari**	*Gruppo Bending Forks*
Verificatore	[da assegnare]		*Prof. Tullio Vardanega*
			Prof. Riccardo Cardin

Abstract dei contenuti:
Raccolta delle norme, delle convenzioni e degli strumenti che regolano il modo di lavorare (*way of working*) del gruppo Bending Forks per tutta la durata del progetto didattico.

Indice

1. Registro delle versioni 3

2. Introduzione 3

2.1. Scopo del documento 3

2.2. Natura incrementale 3

2.3. Scopo del prodotto 3

2.4. Struttura del documento 3

2.5. Glossario 4

2.6. Riferimenti 4

2.6.1. Riferimenti normativi 4

2.6.2. Riferimenti informativi 4

3. Processi primari 4

3.1. Fornitura 4

3.1.1. Attività 4

3.1.2. Rapporti con la proponente 4

3.1.3. Revisioni di avanzamento 5

3.1.4. Documentazione fornita 5

3.2. Sviluppo 6

3.2.1. Analisi dei requisiti 7

3.2.2. *Proof of Concept* 7

3.2.3. Progettazione 8

3.2.4. Codifica 8

4. Processi di supporto 8

4.1. Documentazione 8

4.1.1. Strumenti 8

4.1.2. Struttura comune dei documenti 8

4.1.3. Verbali 9

4.1.4. Diari di bordo 9

4.1.5. Nomi dei file e posizione nella *repository* 9

4.1.6. Versionamento dei documenti 10

4.1.7. Ciclo di vita di un documento 10

4.1.8. Norme tipografiche 10

4.2. Gestione della configurazione 11
4.2.1. *Repository* 11
4.2.2. *Issue* 11
4.2.3. *Branch* e *pull request* 11
4.2.4. Messaggi di *commit* 11
4.3. Accertamento della qualità 12
4.3.1. Strumenti 12
4.3.2. Attività 13
4.4. Verifica 13
4.4.1. Regole generali 13
4.4.2. Verifica dei documenti 13
4.4.3. Analisi statica 13
4.4.4. Analisi dinamica 14
4.5. Validazione 15

5. Processi organizzativi 15

5.1. Gestione dei processi 15
5.1.1. Ruoli 15
5.1.2. Rotazione dei ruoli 16
5.1.3. Ciclo di vita delle attività 16
5.1.4. Rendicontazione delle ore 17
5.1.5. Riunioni 17
5.1.6. Comunicazioni 17
5.2. Infrastruttura 17
5.3. Miglioramento 17
5.4. Formazione 18

6. Metriche di qualità 18

6.1. Metriche di processo 18
6.1.1. Fornitura 18
6.1.2. Documentazione 19
6.1.3. Verifica 20
6.1.4. Gestione della qualità 20
6.2. Metriche di prodotto 20
6.2.1. Funzionalità 20

1. Registro delle versioni

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026-11-10	Lorenzo Milanese, Marco Beltrame	[da assegnare]	Prima stesura del documento in vista della RTB

2. Introduzione

2.1. Scopo del documento

Questo documento descrive il *way of working*$_G$ del gruppo Bending Forks: le regole, le convenzioni e gli strumenti che ogni componente segue durante il progetto. Lo scopo è che tutti lavorino nello stesso modo, così che documenti e codice siano coerenti tra loro e che ogni decisione sia tracciabile.

Le norme sono vincolanti per tutti i componenti del gruppo. Chi trova una norma poco chiara, sbagliata o superata lo segnala al Responsabile, che ne propone la modifica (vedi Processo di miglioramento).

2.2. Natura incrementale

Le Norme di progetto sono un documento **incrementale**: crescono insieme al progetto. La versione attuale copre le attività necessarie a raggiungere la *Requirements and Technology Baseline*$_G$ (RTB): analisi dei requisiti, *Proof of Concept*$_G$ (PoC), documentazione, verifica e gestione del progetto. Le norme sulla progettazione di dettaglio, sulla codifica del prodotto finale e sui test dell'*MVP*$_G$ verranno completate in vista della *Product Baseline*$_G$ (PB). Le parti ancora da decidere sono segnalate in [rosso tra parentesi quadre].

2.3. Scopo del prodotto

Il gruppo realizza il capitolato **C3 – Fossdot**, proposto da **Uranion EMEA LTD**: uno *smart speaker*$_G$ *open source*$_G$, estensibile e rispettoso della *privacy*. [Breve descrizione del prodotto da allineare con l'Analisi dei requisiti dopo la riunione con la proponente del 13 novembre 2026.]

2.4. Struttura del documento

Il documento segue la classificazione dei processi dello standard **ISO/IEC 12207:1995**, adattata al progetto (*tailoring*$_G$):

- **Processi primari**: fornitura e sviluppo;
- **Processi di supporto**: documentazione, gestione della configurazione, accertamento della qualità, verifica, validazione;
- **Processi organizzativi**: gestione, infrastruttura, miglioramento, formazione.

Ogni processo è descritto indicando scopo, strumenti, attività e norme da rispettare. L'ultima sezione definisce le **metriche di qualità** usate per misurare processi e prodotti.

2.5. Glossario

Per evitare ambiguità, i termini tecnici o specifici del dominio sono raccolti nel documento **Glossario**. Nei documenti, la prima occorrenza di un termine presente nel Glossario è scritta in corsivo con una G a pedice, così: *parola*$_G$. In LaTeX si usa il comando `\gl{parola}`.

2.6. Riferimenti

2.6.1. Riferimenti normativi

- Capitolato d'appalto C3 – Fossdot: `https://www.math.unipd.it/~tullio/IS-1/2026/Progetto/C3-Fossdot.pdf`;
- Regolamento del progetto didattico: `https://www.math.unipd.it/~tullio/IS-1/2026/Dispense/PD1.pdf`.

2.6.2. Riferimenti informativi

- Standard ISO/IEC 12207:1995: `https://www.math.unipd.it/~tullio/IS-1/2009/Approfondimenti/ISO_12207-1995.pdf`;
- Conventional Commits: `https://www.conventionalcommits.org/it/v1.0.0/`;
- Glossario v[x.y.z];
- Verbale interno del 4 novembre 2026 (assegnazione della redazione di questo documento e regole di rotazione dei ruoli). [Da Verificare]

3. Processi primari

3.1. Fornitura

Il processo di fornitura regola il rapporto tra il gruppo (fornitore) e la proponente Uranion EMEA LTD e i committenti Prof. Tullio Vardanega e Prof. Riccardo Cardin: dalla candidatura alla consegna del prodotto, passando per la pianificazione e il controllo dell'avanzamento.

3.1.1. Attività

- **Candidatura**: analisi dei capitolati, scelta e presentazione della candidatura (conclusa con l'aggiudicazione del capitolato C3);
- **Pianificazione**: definizione di attività, tempi, costi e rischi nel Piano di progetto;
- **Esecuzione e controllo**: realizzazione del prodotto e monitoraggio continuo di avanzamento e qualità;
- **Revisione con la proponente**: incontri periodici con Uranion per chiarire i requisiti e mostrare l'avanzamento;
- **Consegna**: consegna del prodotto e della documentazione alle revisioni RTB e PB.

3.1.2. Rapporti con la proponente

- I contatti con Uranion sono tenuti dal **Responsabile** tramite l'indirizzo email del gruppo `bending.forks.swe@gmail.com`;
- Gli incontri si svolgono su **Google Meet**, salvo diverso accordo;

- Ogni incontro con la proponente produce un **verbale esterno** (§4.1.3), che viene inviato alla proponente per la firma;
- [Cadenza degli incontri con la proponente da concordare il 13 novembre 2026.]

3.1.3. Revisioni di avanzamento

Il progetto prevede due revisioni di avanzamento con i committenti, a cui il gruppo si candida con la Lettera di presentazione:

- **RTB** (*Requirements and Technology Baseline*): il gruppo dimostra di aver compreso i requisiti del capitolato e di aver scelto tecnologie adeguate, verificate con un ***Proof of Concept*** (§3.2.2);
- **PB** (*Product Baseline*): il gruppo presenta l'architettura del prodotto e l'**MVP** (*Minimum Viable Product*), cioè la versione del prodotto che soddisfa i requisiti obbligatori.

Prima di ogni revisione il Responsabile controlla che tutti i documenti previsti siano in stato **Approvato** (§4.1.7), che non contengano segnaposto e che siano pubblicati sul sito del gruppo.

3.1.4. Documentazione fornita

Il gruppo consegna ai committenti e alla proponente i seguenti documenti:

Documento	Redattore	Uso	Revisione	Contenuto
Lettera di candidatura	Responsabile	Esterno	Candidatura	Candidatura del gruppo al capitolato
Valutazione dei capitolati	Analista	Esterno	Candidatura	Analisi di ogni capitolato e motivazione della scelta
Preventivo costi e assunzione impegni	Responsabile	Esterno	Candidatura	Ore e costi per ruolo e per componente
Lettera di presentazione	Responsabile	Esterno	RTB, PB	Candidatura alla revisione
Analisi dei requisiti	Analista	Esterno	RTB, PB	Casi d'uso e requisiti del prodotto
Piano di progetto	Responsabile	Esterno	RTB, PB	Rischi, pianificazione, preventivi e consuntivi per periodo
Piano di qualifica	Amministrator	Esterno	RTB, PB	Obiettivi di qualità, valori delle metriche, test e loro esiti
Norme di progetto	Amministrator	Interno	RTB, PB	Questo documento
Glossario	Amministrator	Esterno	RTB, PB	Definizione dei termini usati nei documenti
Verbali interni	Responsabile	Interno	RTB, PB	Resoconto delle riunioni interne
Verbali esterni	Responsabile	Esterno	RTB, PB	Resoconto delle riunioni con soggetti esterni
Specifica tecnica	Progettista	Esterno	PB	Architettura e progettazione del prodotto
Manuale utente	Progettista	Esterno	PB	Istruzioni d'uso del prodotto

Oltre ai documenti, il gruppo consegna il **PoC** alla RTB e l'**MVP** alla PB. I documenti consegnati alla RTB vengono aggiornati e riconsegnati alla PB.

[Confermare la tabella, i ruoli redattori e l'elenco delle consegne con il regolamento del progetto didattico dell'A.A. 2026/27.]

3.2. Sviluppo

Il processo di sviluppo comprende le attività che portano dal capitolato al prodotto: analisi dei requisiti, progettazione, codifica, integrazione e test.

In vista della RTB le attività principali sono l'**analisi dei requisiti** e la realizzazione del ***Proof of Concept***. La progettazione di dettaglio e la codifica dell'MVP sono attività della PB: le relative

norme verranno completate prima del loro inizio.

3.2.1. Analisi dei requisiti

L'analisi dei requisiti individua cosa il prodotto deve fare, a partire dal capitolato e dagli incontri con la proponente. Il risultato è il documento **Analisi dei requisiti**.

3.2.1.1. Casi d'uso

Ogni caso d'uso è identificato dal codice:

`UC[Numero].[Sottocaso] - [Titolo]`

dove `[Numero]` identifica il caso d'uso principale, `[Sottocaso]` (facoltativo, anche su più livelli) identifica un caso d'uso derivato e `[Titolo]` riassume l'azione.

Ogni caso d'uso riporta: attori primari e secondari, precondizioni, postcondizioni, scenario principale, eventuali inclusioni ed estensioni, e il diagramma UML dei casi d'uso.

3.2.1.2. Requisiti

Ogni requisito è identificato dal codice:

`R[Tipo]-[Numero]-[Priorità]`

dove:

- `[Tipo]`: `F` funzionale, `Q` di qualità, `V` di vincolo, `P` prestazionale;
- `[Numero]`: progressivo univoco all'interno del tipo;
- `[Priorità]`: `Obb` obbligatorio, `Des` desiderabile, `Opz` opzionale.

Ogni requisito indica la sua **fonte** (capitolato, caso d'uso, verbale) per garantire il tracciamento. [Nomenclatura da allineare con il sotto-gruppo dell'Analisi dei requisiti.]

3.2.1.3. Tracciamento

L'Analisi dei requisiti contiene le tabelle di tracciamento:

- **Fonte → requisiti**: per ogni fonte (capitolato, caso d'uso, verbale) i requisiti che ne derivano;
- **Requisito → fonti**: per ogni requisito le fonti da cui deriva.

Ogni requisito deve avere almeno una fonte e ogni caso d'uso deve essere coperto da almeno un requisito. In vista della PB il tracciamento verrà esteso ai test (§4.5).

3.2.1.4. Strumenti

[Strumento per i diagrammi UML dei casi d'uso da scegliere (es. draw.io, StarUML, PlantUML).]

3.2.2. *Proof of Concept*

Il ***Proof of Concept*** (PoC) è un prototipo, anche parziale e non definitivo, che dimostra che le tecnologie scelte sono adatte a soddisfare i requisiti principali del capitolato. Fa parte della consegna RTB.

- Il PoC si sviluppa in una *repository* dedicata dell'account del gruppo (§4.2.1);

- La *repository* contiene un file `README.md` che spiega cosa dimostra il PoC, quali tecnologie usa e come avviarlo;
- Le tecnologie valutate e i motivi della scelta vengono riportati [nell'Analisi dei requisiti / in un documento apposito / nel Piano di progetto: da decidere];
- Il codice del PoC segue, per quanto possibile, le norme di codifica (§3.2.4) e passa comunque dalla verifica tramite *pull request*.

[Obiettivi del PoC da definire dopo la riunione con la proponente del 13 novembre 2026.]

3.2.3. Progettazione

[Da redigere prima dell'inizio della progettazione (PB): notazione dei diagrammi (UML), livelli di progettazione, *design pattern*, convenzioni per la Specifica tecnica.]

3.2.4. Codifica

Norme valide fin dal PoC:

- Nomi di variabili, funzioni, classi e file, e commenti nel codice, sono in **inglese**;
- Ogni *repository* di codice ha un `README.md` con lo scopo e le istruzioni di avvio;
- Non si caricano nella *repository* file generati, dipendenze scaricate o dati sensibili (password, chiavi, *token*): vanno esclusi con il file `.gitignore`.

[Da completare dopo la scelta delle tecnologie: linguaggi, convenzioni di nomenclatura per ciascun linguaggio, formattatori e linter, struttura delle cartelle.]

4. Processi di supporto

4.1. Documentazione

Il processo di documentazione stabilisce come si scrivono, verificano, pubblicano e aggiornano i documenti del progetto.

4.1.1. Strumenti

- **LaTeX**: linguaggio con cui sono scritti tutti i documenti;
- **GitHub**: *repository* `bendingforksswe/documenti-swe`, che contiene i sorgenti `.tex` ed è la versione di riferimento dei documenti;
- **GitHub Pages**: sito `https://bendingforksswe.github.io/`, che compila e pubblica in automatico i documenti presenti nella *repository*.

4.1.2. Struttura comune dei documenti

Ogni documento parte dal *template* del gruppo e contiene, in quest'ordine:

1. **Copertina**: logo e email del gruppo, titolo, Responsabile, redattori, Verificatore, versione, tipo (interno o esterno), destinatari e un breve *abstract*;
2. **Indice**;
3. **Registro delle versioni**: una riga per versione con versione, data, autore, verificatore e descrizione della modifica, dalla più recente alla più vecchia;

4. **Contenuto**.

L'intestazione di pagina riporta il nome del gruppo a sinistra e il titolo del documento a destra; il piè di pagina riporta il numero di pagina.

4.1.3. Verbali

I verbali documentano le riunioni e le decisioni prese. Si distinguono in:

- **Verbali interni**: riunioni tra i soli componenti del gruppo;
- **Verbali esterni**: riunioni con la proponente o con altri soggetti esterni. Devono essere firmati da un referente esterno.

I verbali si scrivono a partire dal *template* in `template/` e contengono:

- **Informazioni generali**: data, ora di inizio e fine, luogo o piattaforma, tabella dei partecipanti con ruolo e presenza;
- **Ordine del giorno**: elenco numerato dei punti da trattare;
- **Discussione e decisioni**: per ogni punto, la discussione, le decisioni prese (con codice identificativo), la motivazione, gli incaricati e la scadenza;
- **Azioni da intraprendere**: tabella con descrizione, incaricato, scadenza e priorità;
- **Pianificazione della prossima riunione**;
- **Firme**: dello scriba e del verificatore; nei verbali esterni anche del referente esterno.

Il verbale è redatto dallo scriba designato nella riunione e va completato entro [n] giorni dalla riunione.

4.1.4. Diari di bordo

I diari di bordo sono le brevi presentazioni sull'avanzamento del progetto esposte durante le lezioni del Prof. Vardanega. Sono preparati dal Responsabile in carica e caricati in PDF nella cartella `diario-di-bordo/`. [Struttura delle slide da definire.]

4.1.5. Nomi dei file e posizione nella *repository*

La posizione e il nome del file determinano come il documento appare sul sito. Le regole sono:

Percorso	Contenuto
`<fase>/<nome_documento>.tex`	Documenti della fase (`candidatura`, `rtb`, `pb`)
`<fase>/verbali/interni/AAAA-MM-GG_verbale_interno_<n>.tex`	Verbali interni
`<fase>/verbali/esterni/AAAA-MM-GG_verbale-esterno-<azienda>.tex`	Verbali esterni
`.../firmati/<stesso-nome>-firmato.pdf`	PDF firmato di un verbale
`diario-di-bordo/AAAA-MM-GG-diario-di-bordo.pdf`	Diari di bordo
`assets/`	Loghi, firme e immagini
`template/`	*Template* dei documenti

Regole generali:

- I nomi dei file sono in minuscolo, senza spazi né accenti;
- Le date nei nomi dei file sono sempre nel formato `AAAA-MM-GG`, con mese e giorno a due cifre (es. `2026-11-02`, non `2026-11-2`), altrimenti il sito non riconosce la data;
- Il nome del file non contiene la versione: la versione si trova solo nella copertina e nel registro delle versioni.

4.1.6. Versionamento dei documenti

Ogni documento ha una versione nel formato `X.Y.Z`:

- `X` (*major*): aumenta quando il documento viene **approvato** dal Verificatore per una revisione. Finché `X = 0` il documento è in bozza;
- `Y` (*minor*): aumenta quando si aggiungono o modificano contenuti in modo sostanziale (es. una nuova sezione);
- `Z` (*patch*): aumenta per correzioni di refusi, formattazione o piccole modifiche.

Quando un numero aumenta, quelli alla sua destra tornano a zero (es. da `0.2.3` si passa a `0.3.0`). Ogni nuova versione va registrata nel registro delle versioni.

[Da approvare in riunione: i documenti della candidatura usano il formato a due cifre (es. 1.4); valutare se adottare `X.Y.Z` per tutti i documenti RTB.]

4.1.7. Ciclo di vita di un documento

Un documento passa per i seguenti stati:

1. **In redazione**: il redattore scrive o modifica il documento;
2. **In verifica**: il redattore ha finito e un Verificatore controlla il documento;
3. **Verificato**: il Verificatore ha approvato le modifiche, che entrano nel ramo principale della *repository*;
4. **Approvato**: il Responsabile dichiara il documento pronto per una revisione ufficiale (aumento della versione *major*).

Ogni passaggio di stato corrisponde a una versione: una versione `0.Y.Z` indica un documento in lavorazione o verificato ma non ancora approvato; la prima versione `1.0.0` indica il documento approvato per la RTB, la prima `2.0.0` quello approvato per la PB.

I documenti incrementali (Norme di progetto, Piano di progetto, Piano di qualifica, Glossario) possono avere sezioni ancora vuote: in questo caso si verificano e approvano solo le sezioni complete.

4.1.8. Norme tipografiche

- **Date**: nel testo e nelle tabelle si usa il formato `AAAA-MM-GG` (ISO 8601); nel testo discorsivo è ammessa la forma estesa (es. “4 novembre 2026”);
- **Orari**: formato 24 ore `HH:MM`;
- **Titoli**: solo la prima lettera maiuscola, salvo nomi propri (es. “Norme di progetto”, “Analisi dei requisiti”);
- **Elenchi puntati**: ogni voce inizia con la maiuscola e termina con il punto e virgola, tranne l’ultima che termina con il punto;
- **Grassetto**: concetti chiave e termini definiti in un elenco;

- **Corsivo**: termini del Glossario, parole straniere ed enfasi. Non vanno in corsivo i nomi propri di strumenti e servizi (es. GitHub, Discord), le sigle (es. RTB, PoC) e le parole straniere ormai entrate nell'uso italiano (es. file, test, link, email, software);
- **Monospazio**: nomi di file, percorsi, comandi, codice;
- **Ruoli**: sempre con la maiuscola (Responsabile, Amministratore, Analista, Progettista, Programmatore, Verificatore);
- **Riferimenti**: ogni riferimento a un documento indica la versione;
- **Segnaposto**: le parti ancora da completare si scrivono con `\ph{...}`, che le evidenzia in rosso. Un documento da consegnare non deve contenerne.

4.2. Gestione della configurazione

La gestione della configurazione serve a tenere traccia di ogni versione di documenti e codice, di chi l'ha modificata e perché.

4.2.1. *Repository*

- `bendingforksswe/documenti-swe`: sorgenti di tutta la documentazione;
- `bendingforksswe/bendingforksswe.github.io`: sito del gruppo, che pubblica i documenti;
- `bendingforksswe/`[nome]: codice del Proof of Concept, [da creare all'inizio del PoC];
- La *repository* dell'MVP verrà creata all'inizio della PB.

4.2.2. *Issue*

Ogni attività da svolgere viene aperta come ***issue*** su GitHub nella *repository* di riferimento. Ogni *issue* indica:

- Un titolo nel formato `[Documento] attività` (es. `[NdP] stesura processi di supporto`);
- L'assegnatario, unico incaricato dell'attività;
- Il ruolo con cui si svolge l'attività [tramite label o campo del Project];
- La scadenza e, se utile, una descrizione.

[Sigle dei documenti: NdP, AdR, PdP, PdQ, Gloss, ST, MU, Verb. Da confermare.]

4.2.3. *Branch* e *pull request*

[Proposta da approvare in riunione:]

- Il ramo `main` contiene solo materiale verificato e non accetta *push* diretti;
- Ogni attività si svolge su un ramo dedicato, chiamato `issue-<numero>-<breve-descrizione>`;
- Terminata l'attività, il redattore apre una ***pull request*** verso `main`, collegata all'*issue*, e indica il Verificatore come revisore;
- Il Verificatore approva e unisce la *pull request* (*merge*); l'*issue* si chiude e il ramo viene eliminato.

4.2.4. Messaggi di *commit*

I messaggi di *commit* seguono lo standard **Conventional Commits**, con la descrizione in italiano. Il formato è:

```
<tipo>(<ambito>): <descrizione>
```

4.2.4.1. Tipo

Indica la natura della modifica ed è scelto tra:

- `docs`: aggiunta o modifica di contenuti di un documento;
- `fix`: correzione di un errore (refusi e errori di formattazione in un documento, *bug* nel codice);
- `feat`: nuova funzionalità del codice (PoC, MVP, sito);
- `refactor`: modifica del codice che non ne cambia il comportamento;
- `test`: aggiunta o modifica di test;
- `chore`: manutenzione che non tocca contenuti né funzionalità (configurazioni, *template*, struttura delle cartelle, file `.gitignore`).

4.2.4.2. Ambito

Indica cosa viene modificato. Per i documenti è la sigla del documento in minuscolo, la stessa usata nel titolo delle *issue* (es. `ndp`, `adr`, `pdp`, `pdq`); per verbali e diari di bordo si usano `verbali` e `diario`. L'ambito può essere omesso solo se la modifica riguarda l'intera *repository*. [Ambiti per il codice da definire insieme alla struttura del PoC.]

4.2.4.3. Descrizione

- È in italiano, alla terza persona del presente (es. `aggiunge`, `corregge`, `rimuove`);
- Inizia con la lettera minuscola e non termina con il punto;
- È breve (al massimo 72 caratteri) e dice **cosa** cambia, non solo che qualcosa è cambiato: messaggi come `bug fix` o `modifiche` non sono ammessi.

Se serve spiegare il perché della modifica, si aggiunge un corpo dopo una riga vuota.

4.2.4.4. Regole generali

- Ogni *commit* contiene una sola modifica logica: modifiche indipendenti vanno in *commit* separati;
- Il collegamento con l'*issue* si fa nella *pull request*, non nel messaggio di *commit*.

Esempi:

- `docs(ndp): aggiunge sezione sulla verifica`;
- `fix(ndp): corregge refusi nel registro delle versioni`;
- `docs(verbali): aggiunge verbale interno del 2026-11-04`;
- `chore: aggiorna il file .gitignore`.

4.3. Accertamento della qualità

L'accertamento della qualità garantisce che i processi seguiti e i prodotti realizzati rispettino gli obiettivi di qualità fissati dal gruppo.

4.3.1. Strumenti

- **Metriche di qualità**: definite in questo documento (§6), con codice, descrizione e formula;

- **Piano di qualifica**: fissa per ogni metrica il valore **accettabile** e quello **ottimale** e raccoglie i valori misurati;
- **Cruscotto di valutazione**: grafici e tabelle nel Piano di qualifica che mostrano l'andamento delle metriche nel tempo.

4.3.2. Attività

1. A fine periodo l'Amministratore calcola le metriche di processo (§6.1) e di prodotto (§6.2) applicabili;
2. I valori vengono riportati nel Piano di qualifica e confrontati con gli obiettivi;
3. Le metriche fuori dal valore accettabile vengono discusse nella retrospettiva di fine periodo e generano azioni di miglioramento (§5.3).

4.4. Verifica

La verifica controlla che ogni prodotto (documento o codice) sia corretto e rispetti le norme: risponde alla domanda "stiamo costruendo il prodotto nel modo giusto?".

4.4.1. Regole generali

- Ogni prodotto è verificato da un componente **diverso dal redattore**;
- Il Verificatore firma la verifica inserendo il proprio nome nel registro delle versioni;
- Nessun prodotto entra nel ramo principale senza essere stato verificato.

4.4.2. Verifica dei documenti

Il Verificatore legge le parti modificate e controlla:

- Ortografia, grammatica e chiarezza;
- Rispetto delle norme tipografiche (§4.1.8) e della struttura del documento;
- Correttezza e completezza dei contenuti rispetto all'*issue*;
- Che i link e i riferimenti funzionino e indichino la versione;
- Che il documento compili senza errori e non contenga segnaposto se destinato alla consegna.

Se trova errori, li segnala al redattore (come commenti alla *pull request*); il redattore corregge e la verifica si ripete.

4.4.3. Analisi statica

L'analisi statica verifica un prodotto **senza eseguirlo**. Si applica sia ai documenti sia al codice, con due metodi di lettura:

- *Walkthrough*$_{\text{G}}$: lettura completa e a largo spettro, senza sapere in anticipo quali errori cercare. È costosa, quindi si usa per i documenti nuovi o riscritti in gran parte;
- *Inspection*$_{\text{G}}$: lettura mirata, guidata da una ***checklist*** degli errori più frequenti. È più rapida e si usa per le modifiche successive.

Gli errori trovati più spesso durante i *walkthrough* vengono aggiunti alla *checklist* (§4.4.3.1), così che le verifiche successive possano usare l'*inspection*.

4.4.3.1. Checklist *dei documenti*

Durante l'*inspection* di un documento il Verificatore controlla almeno i punti seguenti. Quando segnala un errore nella *pull request*, ne indica il codice (es. `LC-04`).

Codice	Controllo
LC-01	Il documento compila senza errori
LC-02	La versione in copertina coincide con l'ultima riga del registro delle versioni, che descrive la modifica e indica autore e Verificatore
LC-03	Le date sono nel formato `AAAA-MM-GG`, o in forma estesa nel testo discorsivo (§4.1.8)
LC-04	Le parole straniere sono in corsivo, salvo nomi propri, sigle e parole entrate nell'uso italiano
LC-05	Le voci degli elenchi iniziano con la maiuscola e terminano con il punto e virgola, tranne l'ultima che termina con il punto
LC-06	I titoli hanno la maiuscola solo sulla prima lettera, salvo nomi propri
LC-07	I ruoli sono scritti con la maiuscola
LC-08	La prima occorrenza dei termini del Glossario è segnata con `\gl{}`
LC-09	Nomi di file, percorsi e codice sono in monospazio
LC-10	I link funzionano e i rimandi interni (§) puntano alla sezione giusta
LC-11	Non ci sono errori di ortografia né frasi poco chiare
LC-12	Se il documento è destinato alla consegna, non contiene segnaposto

La *checklist* viene aggiornata dall'Amministratore quando nelle verifiche emergono errori ricorrenti non ancora presenti. [*Checklist* per il codice da aggiungere insieme alle norme di codifica.]

Per il codice l'analisi statica comprende anche l'uso di strumenti automatici (*linter*$_{\text{G}}$, analizzatori statici). [Strumenti da scegliere insieme alle tecnologie.]

4.4.4. Analisi dinamica

L'analisi dinamica verifica il codice **eseguendolo**, tramite test. I test devono essere **ripetibili** (stesso risultato a parità di codice) e, dove possibile, **automatici**. Si distinguono:

- **Test di unità**: verificano la più piccola parte di codice verificabile da sola (funzione, classe, modulo);
- **Test di integrazione**: verificano che più unità funzionino correttamente insieme;
- **Test di sistema**: verificano che il sistema completo soddisfi i requisiti dell'Analisi dei requisiti;
- **Test di regressione**: ripetono i test esistenti dopo una modifica, per controllare che non abbia rotto parti già funzionanti;
- **Test di accettazione**: verificano, insieme alla proponente, che il prodotto soddisfi i requisiti del capitolato.

4.4.4.1. Nomenclatura dei test

Ogni test è identificato dal codice:

`T[Tipo]-[Numero]`

dove `[Tipo]` vale `U` (unità), `I` (integrazione), `S` (sistema) o `A` (accettazione) e `[Numero]` è un progressivo univoco all'interno del tipo. I test di regressione non hanno un codice proprio, perché consistono nella ripetizione di test già esistenti.

4.4.4.2. Stato dei test

Ogni test si trova in uno dei seguenti stati: **NI** (non implementato), **I** (implementato, non ancora eseguito), **S** (superato), **NS** (non superato). [Da concordare quando il progetto verrà avviato.]

[Framework di test e strumento per la copertura del codice da scegliere insieme alle tecnologie. Per il PoC sono sufficienti test di unità sulle parti principali.]

4.5. Validazione

La validazione controlla che il prodotto soddisfi le aspettative della proponente: risponde alla domanda "stiamo costruendo il prodotto giusto?".

- **Fino alla RTB**: la validazione riguarda i requisiti. I casi d'uso e i requisiti dell'Analisi dei requisiti vengono presentati alla proponente negli incontri esterni, che ne conferma la correttezza; le conferme e le correzioni sono riportate nei verbali esterni;
- **Verso la PB**: ogni requisito viene collegato ai test di sistema che lo verificano. Un requisito è soddisfatto quando tutti i suoi test sono superati. I test di accettazione si svolgono con la proponente sull'MVP.

5. Processi organizzativi

5.1. Gestione dei processi

5.1.1. Ruoli

Ogni componente ricopre a rotazione tutti i ruoli previsti dal progetto. Il costo orario di ogni ruolo è quello indicato nel Preventivo costi e assunzione impegni.

Ruolo	Compiti principali	Presenza	€/h
Responsabile	Coordina il gruppo, pianifica le attività e le assegna, gestisce tempi, costi e rischi, redige il Piano di progetto, rappresenta il gruppo verso l'esterno, approva i documenti	Tutto il progetto	30
Amministratore	Gestisce l'ambiente di lavoro e gli strumenti, redige e mantiene le Norme di progetto e il Piano di qualifica, calcola le metriche	Tutto il progetto	20
Analista	Studia il dominio e il capitolato, individua casi d'uso e requisiti, redige l'Analisi dei requisiti	Soprattutto fino alla RTB	25
Progettista	Sceglie le tecnologie, definisce l'architettura del prodotto a partire dai requisiti, redige la Specifica tecnica	PoC, poi soprattutto PB	25
Programmatore	Scrive il codice secondo la progettazione e i relativi test	PoC, poi soprattutto PB	15
Verificatore	Verifica documenti e codice secondo queste norme	Tutto il progetto	15

5.1.2. Rotazione dei ruoli

- Dal 16 novembre 2026 i ruoli ruotano ogni **due settimane** (verbale interno del 4 novembre 2026);
- L'assegnazione avviene nella riunione interna di inizio periodo, tenendo conto delle ore già svolte da ciascuno in ogni ruolo, così che a fine progetto la ripartizione rispetti il preventivo;
- Una persona può ricoprire più ruoli nello stesso periodo, ma non può verificare un prodotto di cui è redattore.

5.1.3. Ciclo di vita delle attività

Il Responsabile suddivide il lavoro del periodo in attività piccole, assegnabili a una sola persona e completabili entro il periodo. Ogni attività è una *issue* (§4.2.1) e attraversa i seguenti stati [sulla board del GitHub Project]:

1. ***Backlog***: l'attività è stata individuata ma non ancora pianificata;
2. **Pronta**: il Responsabile l'ha assegnata, con ruolo, ore previste e scadenza;
3. **In corso**: l'assegnatario ci sta lavorando sul proprio ramo;
4. **In verifica**: l'assegnatario ha aperto la *pull request* e indicato il Verificatore;
5. **Completata**: la *pull request* è stata approvata e unita, e l'assegnatario ha registrato le ore effettive.

5.1.3.1. Definition of Done

Un'attività è **completata** solo quando:

- Il prodotto è stato verificato e approvato da un Verificatore diverso dall'assegnatario;
- Le modifiche sono state unite nel ramo `main`;

- L'*issue* è chiusa e riporta le ore effettivamente svolte. [Discussione con il gruppo per l'organizzazione di gestione delle ore.]

5.1.4. Rendicontazione delle ore

Ogni componente registra le ore svolte, indicando ruolo e attività. Il Responsabile le riporta nel Piano di progetto a fine periodo, confrontandole con il preventivo. [Strumento di rendicontazione da scegliere: GitHub Project, foglio Google condiviso o altro.]

5.1.5. Riunioni

- **Riunioni interne**: si svolgono su Discord, di norma all'inizio di ogni periodo, per fare il punto sul lavoro svolto, pianificare il periodo successivo e assegnare i ruoli. [Cadenza fissa da definire];
- **Riunioni esterne**: con la proponente su Google Meet, o con i committenti;
- **Riunioni straordinarie**: convocate dal Responsabile in caso di necessità.

Ogni riunione nomina uno scriba, che redige il verbale.

5.1.6. Comunicazioni

- **Interne**: WhatsApp per le comunicazioni rapide e i sondaggi, Discord per le riunioni e il lavoro sincrono;
- **Esterne**: email dall'indirizzo `bending.forks.swe@gmail.com`, gestita dal Responsabile, e Google Meet per gli incontri con la proponente.

5.2. Infrastruttura

Strumenti adottati dal gruppo:

Strumento	Utilizzo
GitHub	Versionamento di documenti e codice, *issue*, *pull request*, Actions
GitHub Pages	Pubblicazione dei documenti sul sito del gruppo
Discord	Riunioni interne e lavoro sincrono
Gmail	Comunicazioni esterne
Google Calendar	Calendario condiviso di riunioni e scadenze
Google Drive	Condivisione di file di supporto
Google Meet	Incontri con la proponente

L'Amministratore in carica è responsabile del funzionamento degli strumenti e risolve i problemi che emergono.

5.3. Miglioramento

Il gruppo migliora il proprio modo di lavorare con il ciclo $PDCA_G$ (*Plan, Do, Check, Act*):

1. A fine periodo, nella riunione interna, si fa una breve retrospettiva su cosa ha funzionato e cosa no;
2. I problemi emersi diventano proposte di modifica alle norme;
3. Le modifiche approvate vengono inserite in questo documento con una nuova versione;
4. Nel periodo successivo si controlla, anche tramite le metriche del Piano di qualifica, se la modifica ha funzionato.

5.4. Formazione

Ogni componente studia in autonomia gli strumenti e le tecnologie adottate. Quando il gruppo adotta una tecnologia nuova, chi la conosce meglio la presenta agli altri in una breve sessione su Discord. Chi assume un ruolo per la prima volta si confronta con chi lo ha ricoperto in precedenza. Lo sviluppo del PoC è il momento principale di formazione sulle tecnologie scelte.

Risorse per gli strumenti già in uso:

- Git: `https://git-scm.com/docs`;
- GitHub: `https://docs.github.com`.

[Risorse di studio per le tecnologie del PoC, da aggiungere dopo la loro scelta.]

6. Metriche di qualità

[Da capire] Questa sezione stabilisce **cosa** si misura e **come** si calcola. Gli obiettivi (valore accettabile e valore ottimale) e i valori misurati in ogni periodo si trovano nel **Piano di qualifica**.

Ogni metrica è identificata dal codice:

`M[Categoria]-[Sigla]`

dove `[Categoria]` vale `PC` per le metriche di **processo** e `PD` per quelle di **prodotto**, e `[Sigla]` è l'abbreviazione del nome della metrica.

6.1. Metriche di processo

6.1.1. Fornitura

Le metriche di fornitura seguono il metodo dell'*Earned Value*$_{\text{G}}$. Si calcolano a fine periodo usando le ore previste ed effettive delle attività e il costo orario dei ruoli (§5).

Codice	Nome	Descrizione e formula
MPC-BAC	*Budget at Completion*	Costo totale preventivato del progetto, fissato nel Preventivo costi e assunzione impegni
MPC-PV	*Planned Value*	Valore del lavoro pianificato fino alla data di misura: $PV = BAC \times \%\,\text{lavoro pianificato}$
MPC-AC	*Actual Cost*	Costo effettivamente sostenuto: somma delle ore effettive di ogni attività per il costo orario del ruolo
MPC-EV	*Earned Value*	Valore del lavoro effettivamente completato: $EV = BAC \times \%\,\text{lavoro completato}$
MPC-CPI	*Cost Performance Index*	Efficienza dei costi: $CPI = EV/AC$. Se $CPI < 1$ si spende più del previsto
MPC-SPI	*Schedule Performance Index*	Rispetto dei tempi: $SPI = EV/PV$. Se $SPI < 1$ si è in ritardo
MPC-CV	*Cost Variance*	Differenza tra valore prodotto e costo: $CV = EV - AC$
MPC-SV	*Schedule Variance*	Differenza tra lavoro svolto e pianificato: $SV = EV - PV$
MPC-EAC	*Estimate at Completion*	Stima del costo finale: $EAC = BAC/CPI$
MPC-ETC	*Estimate to Complete*	Stima del costo per terminare il progetto: $ETC = EAC - AC$
MPC-RNP	Rischi non previsti	Numero di problemi verificatisi nel periodo che non erano presenti nell'analisi dei rischi del Piano di progetto

6.1.2. Documentazione

Codice	Nome	Descrizione e formula
MPC-IG	Indice di Gulpease	Leggibilità di un testo in italiano, su una scala da 0 a 100 (più alto è più leggibile): $IG = 89 + \frac{300 \times \text{frasi} - 10 \times \text{lettere}}{\text{parole}}$
MPC-CO	Correttezza ortografica	Numero di errori ortografici e grammaticali trovati in verifica in un documento

[Strumento per il calcolo automatico dell'indice di Gulpease e per il controllo ortografico da scegliere.]

6.1.3. Verifica

Codice	Nome	Descrizione e formula
MPC-TS	Test superati	Percentuale di test superati sul totale dei test eseguiti: $\frac{\text{test superati}}{\text{test eseguiti}} \times 100$
MPC-CC	*Code coverage*	Percentuale di righe di codice eseguite dai test: $\frac{\text{righe eseguite}}{\text{righe totali}} \times 100$

6.1.4. Gestione della qualità

Codice	Nome	Descrizione e formula
MPC-MS	Metriche soddisfatte	Percentuale di metriche con valore almeno accettabile: $\frac{\text{metriche accettabili}}{\text{metriche misurate}} \times 100$

6.2. Metriche di prodotto

Le metriche di prodotto si applicano al codice: alla RTB solo in parte, sul PoC; per intero sull'MVP.

6.2.1. Funzionalità

Codice	Nome	Descrizione e formula
MPD-ROS	Requisiti obbligatori soddisfatti	$\frac{\text{requisiti obbligatori soddisfatti}}{\text{requisiti obbligatori totali}} \times 100$
MPD-RDS	Requisiti desiderabili soddisfatti	$\frac{\text{requisiti desiderabili soddisfatti}}{\text{requisiti desiderabili totali}} \times 100$
MPD-RZS	Requisiti opzionali soddisfatti	$\frac{\text{requisiti opzionali soddisfatti}}{\text{requisiti opzionali totali}} \times 100$

Un requisito è soddisfatto quando tutti i test di sistema collegati sono superati (§4.5).

[Metriche di manutenibilità, affidabilità, usabilità ed efficienza (es. complessità ciclomatica, branch coverage, tempo di risposta) da definire in vista della PB, in base alle tecnologie scelte e ai requisiti di qualità dell'Analisi dei requisiti.]